

Rapport global d'analyse fonctionnelle et technique

Projet NewsHub

Document de comprehension complet du projet, de son architecture et de ses scenarios d'usage.
L'objectif est qu'une personne qui lit ce rapport comprenne comment l'application fonctionne, comment les composants interagissent et comment chaque fonctionnalite principale est executee.

Fiche d'identite du projet
Date du rapport : 2026-04-28
Nom du produit : NewsHub
Nature : plateforme web d'actualites avec experience personnalisee, premium et live
Public cible : visiteurs, utilisateurs connectes, membres premium, editeurs live

Indicateur	Valeur observee dans le projet
Frontend	Angular standalone components + Router + RxJS + Bootstrap d'appui
Backend	FastAPI + SQLAlchemy + JWT + WebSocket
Base de donnees	MySQL avec creation automatique et seed des centres d'interet
Fonctions metier	authentification, fil d'actualites, favoris, commentaires, premium, IA, live
Temps reel	WebSocket + WebRTC pour les rooms live
IA locale	Ollama + modele prefere qwen3:14b (implementation actuelle)

Resume executif : NewsHub est une application SPA Angular connectee a un backend FastAPI. Elle agrege des actualites externes, personnalise le fil selon les interets choisis lors de l'inscription, permet de sauvegarder des articles, publier des commentaires, activer un statut premium et acceder a un assistant IA local. Le projet contient aussi un module live en temps reel avec WebSocket et WebRTC.

Sommaire

1. Vision generale et objectifs du projet
2. Architecture globale et structure du code
3. Fonctionnalites principales et interactions utilisateur
4. Scenarios metier et diagrammes de fonctionnement
5. Donnees, API, securite et stockage
6. Technologies utilisees et bilan du projet

Lecture conseillée
Commencer par l'architecture pour comprendre les blocs.
Ensuite lire les scenarios utilisateurs pour voir la logique metier.
Enfin consulter les sections API/base de donnees pour la comprehension technique detaillee.

1. Vision generale et objectifs du projet

NewsHub cherche a offrir une experience moderne de consultation d'actualites. Le systeme ne se limite pas a afficher des news : il personnalise l'accueil, gere les comptes, memorise des actions utilisateur et ajoute des fonctions avancees comme le premium, le chatbot local et les salles live.

- Afficher des actualites par categorie a partir d'une source externe.
- Filtrer rapidement les articles grace a un cache local cote frontend.
- Permettre une inscription en deux etapes avec choix d'interets.
- Faire evoluer l'experience selon l'etat utilisateur : visiteur, connecte, premium, editeur.
- Centraliser l'activite de l'utilisateur : profil, favoris, commentaires, premium.
- Ajouter un module live avec communication temps reel et streaming navigateur a navigateur.

Conclusion de la vision

Le projet combine une logique de media, de compte utilisateur, de personnalisation, de paiement simule, d'IA locale et de live streaming.

Cela en fait un projet riche, utile pour montrer une maitrise full-stack au-dela d'un simple site vitrine.

2. Architecture globale et structure du code

L'architecture est decoupee en deux grands sous-systemes : le frontend Angular et le backend FastAPI.

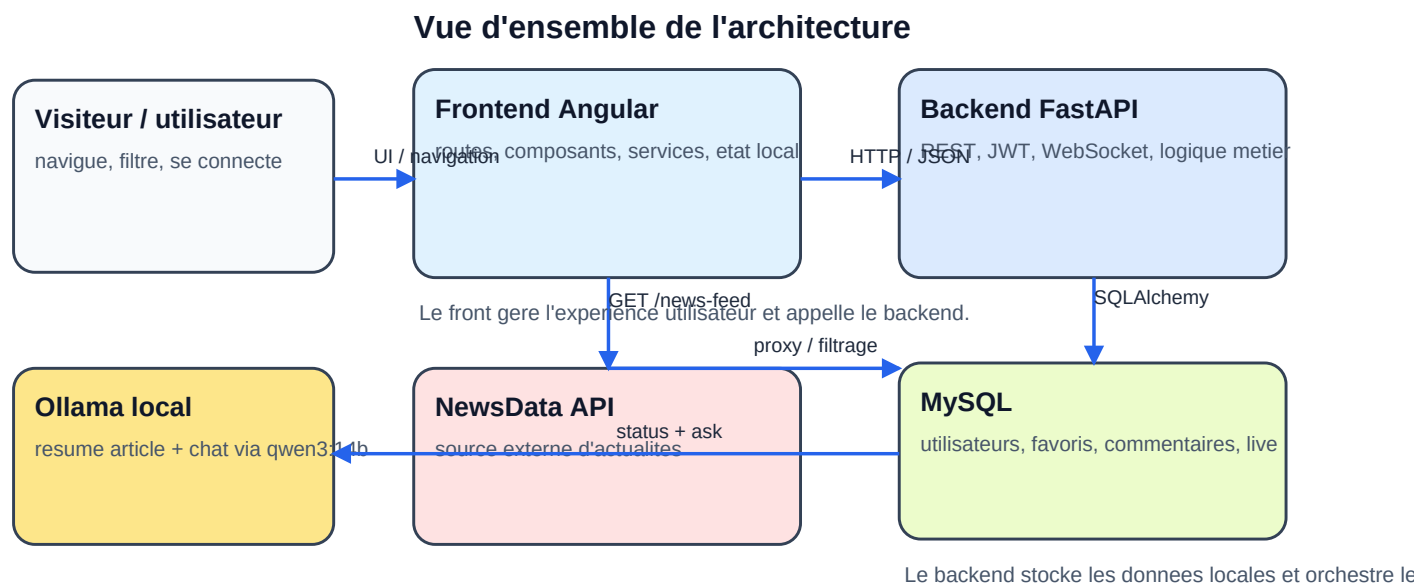


Figure 1 - Architecture globale de NewsHub.

Le frontend pilote l'interface, les routes et les services HTTP. Le backend centralise la logique metier, la securite JWT, l'accès MySQL, les routes live et l'assistant Ollama.

Dossier	Role principal
frontend/src/app/features	Pages metier: home, details, profile, premium, live hub, live room
frontend/src/app/shared/components	Composants reutilisables: header, footer, auth-card, cards, filtres
frontend/src/app/core/services	Services Angular pour news, auth, profil, favoris, commentaires, chatbot, live
backend/main.py	API principale FastAPI, endpoints HTTP et WebSocket
backend/models.py	Modele relationnel SQLAlchemy
backend/database.py	Connexion MySQL, creation automatique, extensions de schema, seed des interets
backend/security.py	Hash des mots de passe et creation/verification des JWT
backend/simple_chatbot.py	Assistant IA local base sur Ollama

3. Fonctionnalités principales et interactions utilisateur

Les principales fonctionnalités peuvent être regroupées en huit modules.

A. Fil d'actualités et filtres

Le service news précharge plusieurs catégories, dédouble les articles, trie par date et persiste le cache dans localStorage. Le Home recompose ensuite l'affichage selon les filtres et les intérêts préférés.

B. Authentification et inscription

L'utilisateur peut s'inscrire en deux étapes : informations d'identité puis choix des intérêts. La vérification d'email est déclenchée juste avant la création finale du compte, puis le backend crée l'utilisateur, lie ses intérêts et renvoie un token JWT.

C. Profil utilisateur

La page profil recharge l'utilisateur depuis l'API, permet la mise à jour du nom, de l'email, du mot de passe et de la photo de profil. Les validations sont faites côté frontend puis confirmées côté backend.

D. Favoris

Depuis la page détail, un utilisateur connecté peut sauvegarder un article. Si l'article n'a pas d'URL source valide, l'action est refusée. Les favoris sont ensuite visibles dans le dashboard profil.

E. Commentaires

Chaque article peut porter une discussion. Les visiteurs sont redirigés vers le login, tandis que les membres connectés postent via /comments.

F. Premium

La page premium simule un paiement. Une fois valide, le backend active is_premium et stocke le plan choisi. Le front rafraîchit ensuite la session utilisateur.

G. Assistant IA

Le panneau IA d'article interroge le backend pour obtenir l'état Ollama, un bref article et des réponses conversationnelles. L'implémentation actuelle utilise qwen3:14b et non gemma3:12b.

H. Live Hub

Un éditeur crée une room live, la démarre, diffuse sa caméra et publie des updates. Les viewers premium rejoignent via WebSocket et WebRTC, peuvent chatter et reçoivent les changements d'état en temps réel.

Parcours utilisateur principaux

- Visiteur : consulte l'accueil, filtre des actualités, ouvre les détails, puis est redirigé vers le login s'il veut sauvegarder, commenter ou utiliser le premium.
- Utilisateur standard : se connecte, retrouve ses intérêts, sauvegarde des articles, commente et peut activer le premium.
- Utilisateur premium : accède en plus à l'assistant IA sur les articles et aux salles live premium.
- Éditeur : crée et administre des rooms live, contrôle le démarrage, l'arrêt et la diffusion caméra.

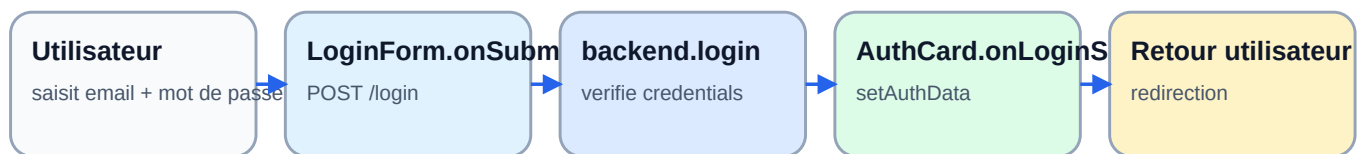
4. Fiches fonctionnelles detaillees

Cette section descend au niveau des fonctions exactes du projet. Pour chaque fonctionnalite, le rapport indique les methodes mobilisees, leur localisation, la logique metier et l'efficacite de la solution dans NewsHub.

4.1 Connexion utilisateur

Fonction / methode	Localisation	Role dans le projet
LoginForm.onSubmit()	frontend/src/app/shared/components/login-form/login-form.ts	Envoie les identifiants au backend et remonte le succes ou l'erreur au composant parent.
AuthCard.onLoginSuccess()	frontend/src/app/shared/components/auth-card/auth-card.ts	Stocke la session via AuthService puis redirige vers returnUrl.
AuthService.setAuthData()	frontend/src/app/core/services/auth/auth.ts	Met a jour localStorage et l'observable currentUser\$.
login()	backend/main.py	Verifie l'email, le mot de passe hash et renvoie un JWT + l'utilisateur serialize.
create_access_token()	backend/security.py	Fabrique le token JWT utilise par le frontend.

Diagramme de la fonctionnalite Login



Cette chaine rend la connexion rapide et centralise l'etat de session dans AuthService.

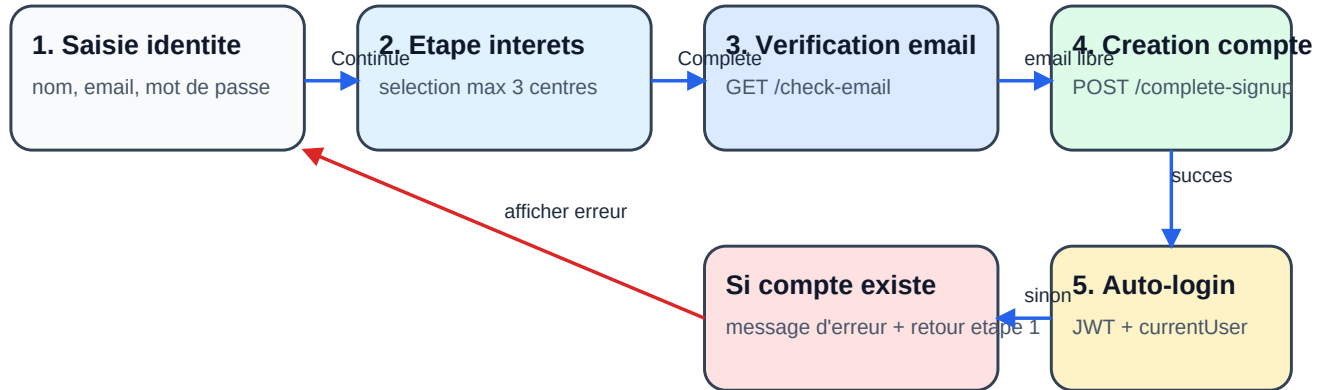
Explication : la connexion est volontairement courte. Le composant LoginForm ne garde pas la logique metier ; il delegue au backend la verification réelle des credentials. L'efficacite de cette conception vient du fait que la validation sensible ne quitte jamais le serveur, tandis que le frontend ne gere que l'affichage et la redirection.

4.2 Inscription et choix des interets

Fonction / methode	Localisation	Role dans le projet
RegisterForm.onSubmit()	frontend/src/app/shared/components/register-form/register-form.ts	Valide les champs locaux et emet les donnees de base vers AuthCard.
AuthCard.goToNextStep()	frontend/src/app/shared/components/auth-card/auth-card.ts	Conserve les informations d'identite et passe a l'etape interets.
InterestsForm.submit()	frontend/src/app/shared/components/interests-form/interests-form.ts	Envoie la liste finale des interets selectionnes.
AuthCard.finishSignup()	frontend/src/app/shared/components/auth-card/auth-card.ts	Verifie d'abord l'email puis poste l'inscription finale.
check_email()	backend/main.py	Teste l'existence d'un compte sur l'email fourni.

Fonction / methode	Localisation	Role dans le projet
complete_signup()	backend/main.py	Cree l'utilisateur, lie les interets et retourne la session authentifiee.

Flux d'inscription reele du projet

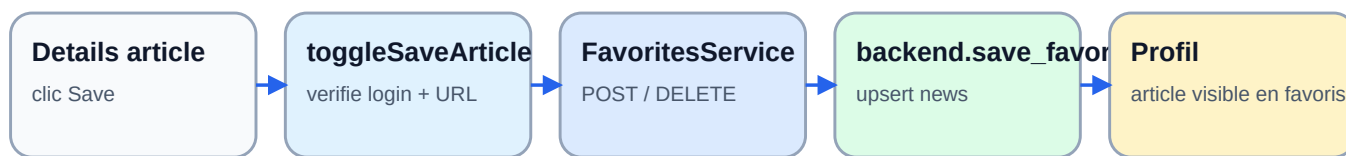


Explication : ce flux en deux etapes ameliore l'onboarding, car il separe la creation d'identite de la personnalisation. Son efficacite tient au fait que le frontend collecte progressivement les informations, tandis que le backend conserve le dernier mot sur l'unicite de l'email et la creation effective du compte.

4.3 Sauvegarde des favoris

Fonction / methode	Localisation	Role dans le projet
NewsDetailsPageComponent.toggleSaveArticle()	frontend/src/app/features/news/news-details-page/news-details-page.ts	Point d'entree UI pour sauvegarder ou retirer un favori.
FavoritesService.saveArticle()	frontend/src/app/core/services/favorites.ts	Mappe l'article frontend vers le payload backend puis appelle POST /favorites.
FavoritesService.removeArticle()	frontend/src/app/core/services/favorites.ts	Supprime un favori via DELETE /favorites.
FavoritesService.isArticleSaved()	frontend/src/app/core/services/favorites.ts	Charge l'etat saved/non saved pour l'article courant.
save_favorite()	backend/main.py	Cree ou remet a jour l'association favorite entre l'utilisateur et la news.
remove_favorite()	backend/main.py	Supprime la liaison entre l'utilisateur et la news.
crud.upsert_news_record()	backend/crud.py	Assure qu'un article existe en base avant d'etre reference comme favori.

Diagramme de la fonctionnalite Save Favorite



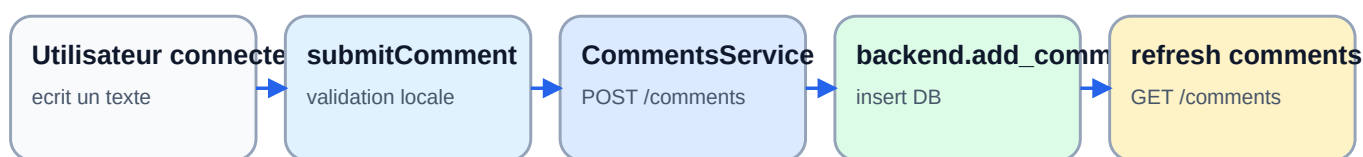
Le design evite les doublons et garantit qu'un favori pointe toujours vers un article persiste.

Explication : l'efficacite vient d'une conception robuste. Avant de stocker un favori, le backend normalise l'article et le persiste si necessaire. Ainsi, le projet evite de multiplier des copies incoherentes et peut reutiliser la meme news pour les commentaires et l'historique utilisateur.

4.4 Commentaires sur les articles

Fonction / methode	Localisation	Role dans le projet
NewsDetailsPageComponent.submitComment()	frontend/src/app/features/news/news-details-page/news-details-page.ts	Valide le texte, verifie la session puis envoie le commentaire.
CommentsService.addComment()	frontend/src/app/core/services/comments.ts	Poste le commentaire et l'article associe.
CommentsService.getComments()	frontend/src/app/core/services/comments.ts	Recharge tous les commentaires d'une URL d'article.
add_comment()	backend/main.py	Nettoie le texte, rattache la news et insere le commentaire.
get_comments()	backend/main.py	Retourne la liste ordonnee des commentaires d'un article.

Diagramme de la fonctionnalite Commentaire



Le refresh immediat donne un retour utilisateur clair et maintient la page coherente.

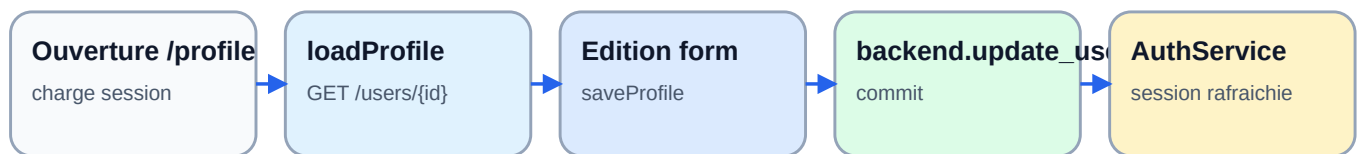
Explication : le commentaire reutilise la meme structure d'article que les favoris. Cela augmente l'efficacite du projet, car un article deja persiste peut etre partage entre plusieurs tables metier sans duplication conceptuelle du schema.

4.5 Gestion du profil

Fonction / methode	Localisation	Role dans le projet
ProfilePageComponent.loadProfile()	frontend/src/app/features/profile/profile-page/profile-page.ts	Recharge le profil depuis l'API pour ne pas dependre uniquement de localStorage.

Fonction / methode	Localisation	Role dans le projet
ProfilePageComponent.saveProfile()	frontend/src/app/features/profile/profile-page/profile-page.ts	Construit full_name, email et eventuellement le changement de mot de passe.
ProfilePageComponent.saveProfilePhoto()	frontend/src/app/features/profile/profile-page/profile-page.ts	Envoie la photo encodee et optimisee.
ProfileService.updateProfileDetails()	frontend/src/app/core/services/profile.ts	PUT du profil texte.
ProfileService.updateProfilePhoto()	frontend/src/app/core/services/profile.ts	PUT de la photo.
update_user_profile()	backend/main.py	Verifie l'unicite de l'email et les regles de changement de mot de passe.
update_user_profile_photo()	backend/main.py	Normalise et stocke la photo.

Diagramme de la fonctionnalite Profil



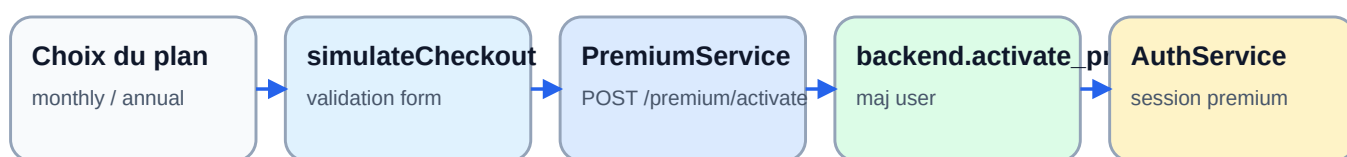
Le profil reste synchronise entre le backend et la session frontend.

Explication : la page profil n'est pas seulement un formulaire. Elle recalcule la completion, recharge la session et separe clairement la photo des details textuels. Cette separation ameliore l'efficacite des interactions et simplifie les validations.

4.6 Premium et paiement simule

Fonction / methode	Localisation	Role dans le projet
PremiumPageComponent.simulateCheckout()	frontend/src/app/features/premium/premium-page/premium-page.ts	Valide les champs puis appelle le service premium.
PremiumService.activatePremium()	frontend/src/app/core/services/premium.ts	POST /premium/activate.
PremiumService.decorateUser()	frontend/src/app/core/services/premium.ts	Normalise les champs premium dans la session frontend.
activate_premium_membership()	backend/main.py	Active is_premium, premium_plan et premium_since.

Diagramme de la fonctionnalite Premium



Le projet simule un paiement sans passer par une vraie passerelle, ce qui est idéal pour la démonstration.

Explication : cette fonctionnalité est efficace pédagogiquement. Elle montre un vrai flux d'abonnement sans dépendance à Stripe ou à une banque, ce qui permet de concentrer l'analyse sur la logique métier et les changements d'état utilisateur.

4.7 Assistant IA premium

Fonction / methode	Localisation	Role dans le projet
ArticleAssistantPanelComponent.loadStatus()	frontend/src/app/features/news/article-assistant-panel/article-assistant-panel.ts	Charge l'état d'Ollama et du modèle.
ArticleAssistantPanelComponent.loadBrief()	frontend/src/app/features/news/article-assistant-panel/article-assistant-panel.ts	Demande un résumé structurant de l'article.
ArticleAssistantPanelComponent.send()	frontend/src/app/features/news/article-assistant-panel/article-assistant-panel.ts	Construit l'historique récent et envoie le prompt.
ChatbotService.getStatus()	frontend/src/app/core/services/chatbot.ts	GET /chatbot/status.
ChatbotService.getArticleBrief()	frontend/src/app/core/services/chatbot.ts	POST /chatbot/article-brief.
ChatbotService.askChatbot()	frontend/src/app/core/services/chatbot.ts	POST /chatbot/ask.
get_chatbot_status()	backend/simple_chatbot.py	Teste Ollama et la disponibilité de qwen3:14b.
get_article_brief()	backend/simple_chatbot.py	Produit un résumé simple à partir du texte de l'article.

Fonction / methode	Localisation	Role dans le projet
ask_chatbot()	backend/simple_chatbot.py	Construit le prompt article-aware et appelle /api/chat d'Ollama.

Diagramme de la fonctionnalite Assistant IA



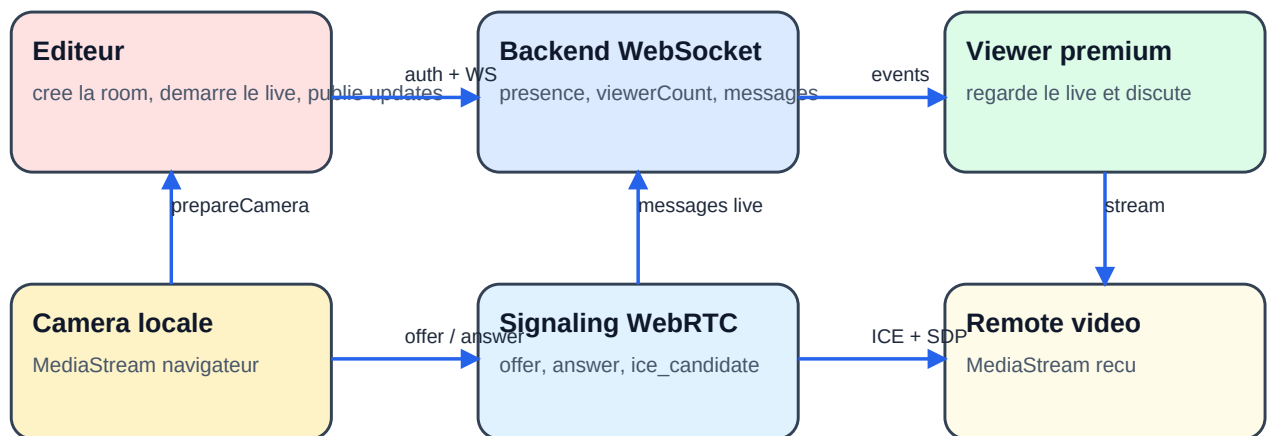
La limite d'historique garde le chat reactif et reduit la charge d'appel du modele.

Explication : l'assistant est efficace parce qu'il reste simple. Il n'utilise pas une stack RAG lourde ; il injecte plutot le texte de l'article dans un prompt systeme puis ajoute les derniers tours de conversation. Cela convient bien a une application locale ou a un projet de demonstration.

4.8 Module live temps reel

Fonction / methode	Localisation	Role dans le projet
LiveHubPageComponent.createLiveRoom()	frontend/src/app/features/live/live-hub-page/live-hub-page.ts	Cree une salle live reservee a l'editeur.
LiveRoomPageComponent.startLiveRoom()	frontend/src/app/features/live/live-room-page/live-room-page.ts	Passe l'evenement au statut live.
LiveRoomPageComponent.goLiveWithCamera()	frontend/src/app/features/live/live-room-page/live-room-page.ts	Prepare la camera et annonce broadcaster_ready.
LiveRoomPageComponent.publishEditorUpdate()	frontend/src/app/features/live/live-room-page/live-room-page.ts	Diffuse un update texte a tous les viewers.
LiveRoomPageComponent.sendChatMessage()	frontend/src/app/features/live/live-room-page/live-room-page.ts	Diffuse un message de chat.
LiveRoomSocketService.connect()	frontend/src/app/core/services/live-room-socket.ts	Ouvre la connexion WebSocket de signaling.
create_live_event(), start_live_event(), end_live_event()	backend/main.py	Pilotent le cycle de vie metier d'une room.
live_event_socket()	backend/main.py	Traite les evenements chat, signaling WebRTC, viewer_count et updates live.
LiveRoomManager.connect()/broadcast()	backend/main.py	Maintient les connexions WebSocket par room.

Architecture du module live temps reel



Le backend ne transporte pas la video elle-meme: il orchestre le signaling et l'etat de la room.

Explication : c'est l'une des parties les plus ambitieuses du projet. L'efficacite repose sur un bon partage des roles : REST pour l'etat metier, WebSocket pour les evenements temps reel, WebRTC pour la video. Ce decoupage est exactement ce qu'on attend d'une architecture live moderne.

5. Scenarios metier et diagrammes de fonctionnement

Cette section traduit le comportement du projet en scenarios lisibles.

Scenario 1 - Inscription complete

- L'utilisateur ouvre /register et remplit nom, email, mot de passe et confirmation.
- Le composant RegisterForm valide la coherence locale des champs.
- Le composant AuthCard passe ensuite a l'ecran de selection des interets.
- Apres selection, AuthCard appelle GET /check-email puis POST /complete-signup si l'adresse est libre.
- Le backend cree le compte, rattache les interets et renvoie un JWT ; le frontend connecte automatiquement l'utilisateur.

Flux d'inscription reele du projet

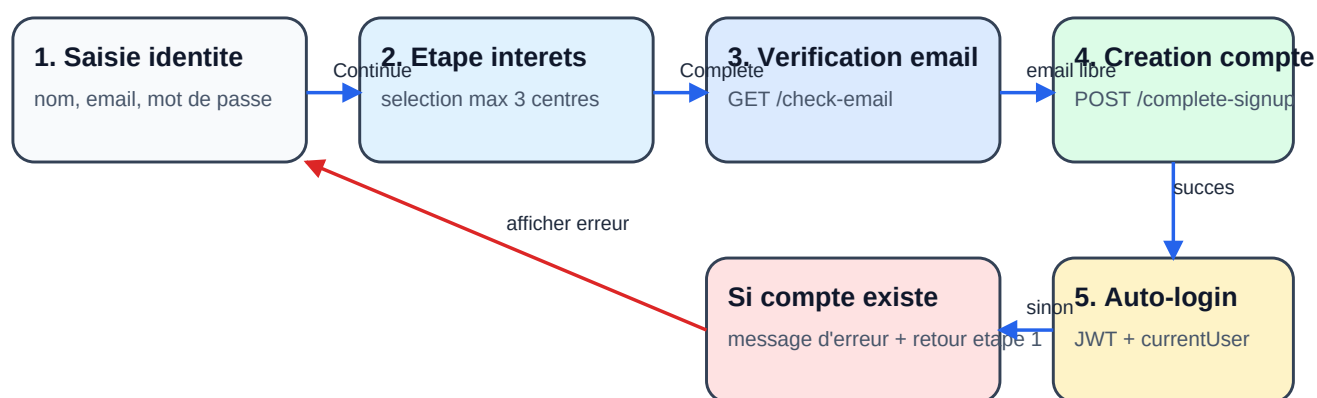
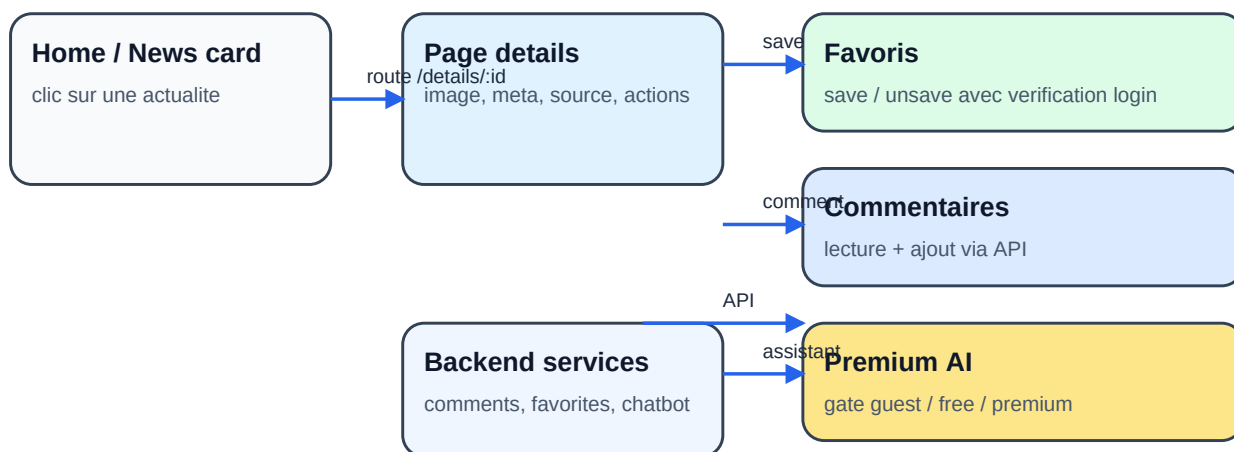


Figure 2 - Scenario d'inscription conforme a l'implementation actuelle.

Scenario 2 - Consultation et exploitation d'un article

Cycle d'interaction autour d'un article



Un meme article devient le point d'entree vers la consultation, la sauvegarde, la discussion et l'assistance IA.

Figure 3 - Cycle d'interaction autour d'un article.

- Depuis le Home, l'utilisateur clique sur une carte d'actualite.
- La page details recharge l'article via l'etat de navigation ou via le store de news.
- A partir de cette page, il peut enregistrer l'article, commenter, consulter la source ou ouvrir l'IA premium.
- Le niveau d'accès dépend de la session : visiteur, standard ou premium.

Scenario 3 - Experience premium et IA

- L'utilisateur active le premium depuis la page /premium avec un paiement simule.
- Le backend met a jour les champs premium de l'utilisateur.
- Sur la page d'un article, le panneau assistant appelle /chatbot/status, /chatbot/article-brief puis /chatbot/ask.
- Le backend interroge Ollama et construit une reponse en injectant le texte de l'article dans le prompt systeme.

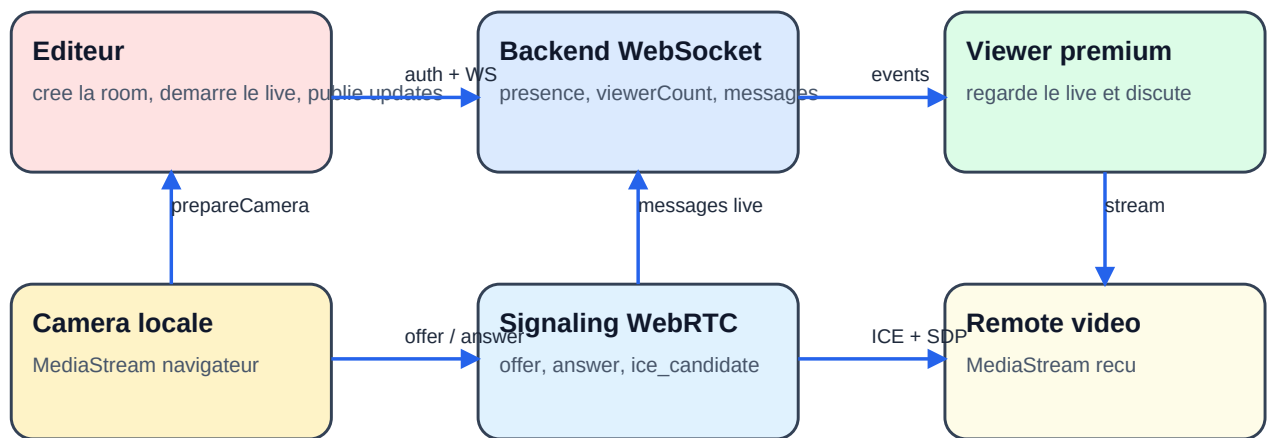
Point d'analyse important

Le brief fonctionnel mentionne gemma3:12b, mais le code reel du backend utilise qwen3:14b dans simple_chatbot.py.

Le rapport doit donc retenir qwen3:14b comme implementation effective actuelle.

Scenario 4 - Module live temps reel

Architecture du module live temps reel



Le backend ne transporte pas la video elle-meme: il orchestre le signaling et l'etat de la room.

Figure 4 - Flux temps reel du module live.

- L'editeur cree un event live depuis /live, puis ouvre la room /live/:id.
- La room utilise des endpoints REST pour l'etat metier et un WebSocket pour la presence, le chat et le signaling.
- La video n'est pas transmise par le backend : elle passe directement entre navigateurs via WebRTC.
- Le backend conserve toutefois les messages live et les updates editeur en base.

6. Donnees, API, securite et stockage

Le systeme s'appuie sur une base MySQL, une securite JWT, du stockage local navigateur et des routes REST/WebSocket.

6.1 Modele de donnees

Table	Role dans l'application
users	Compte utilisateur, role, photo, premium, historique minimal de session
interests	Liste des centres d'interet disponibles
user_interests	Association n-n entre utilisateurs et interets
source	Source de publication d'un article
news	Copie locale des articles qui deviennent favoris ou support de commentaire
favorite	Articles sauvegardes par utilisateur
comments	Commentaires postes sur un article
live_events	Salles live creees par un editeur
live_messages	Messages de chat et updates editeur pendant le live

Le backend cree automatiquement la base si elle n'existe pas et ajoute aussi les colonnes profile_photo, role, premium et premium_since lorsqu'elles manquent. Il seed egalement les interets par default.

6.2 API et contrats d'echange

Groupe	Endpoints cles	Utilite
Auth	GET /check-email, POST /complete-signup, POST /login, GET /users/me	Creation de session, verification email, identification de l'utilisateur
Profil	GET /users/{id}, PUT /users/{id}/profile, PUT/DELETE /users/{id}/profile/photo	Consultation et mise a jour du profil
News	GET /news-feed	Intermediation vers NewsData avec filtres et normalisation
Interets	GET /interests	Chargement des centres d'interet pour l'inscription
Favoris	POST /favorites, DELETE /favorites, GET /favorites-status, GET /favorites/{id}	Sauvegarde et recuperation des articles favoris
Commentaires	GET /comments, POST /comments	Discussion autour d'un article
Premium	POST /premium/activate	Activation premium simulee cote backend
IA	GET /chatbot/status, POST /chatbot/article-brief, POST /chatbot/ask	Etat Ollama, resume article, conversation IA
Live	GET/POST /live-events, POST /live-events/{id}/start, POST /end, DELETE /live-events/{id}, WS /ws/live-events/{id}	Evenements live, chat temps reel, video et updates editeur

6.3 Securite et session

- Le mot de passe est hashe avec bcrypt si disponible, sinon PBKDF2 SHA-256.
- Le backend cree un JWT avec une date d'expiration et le frontend le relit depuis localStorage.

- AuthService supprime automatiquement une session invalide ou expiree.
- Les endpoints sensibles verifient l'utilisateur courant et interdisent l'accès à un autre compte.
- Le rôle editor est exigé pour créer, lancer, terminer ou supprimer une room live.

6.4 Stockage local navigateur

- currentUser : session utilisateur courante decorée par PremiumService.
- access_token : JWT utilisé par les appels protégés.
- newshub-news-store-v1 : cache des catégories d'actualités préchargées.
- Le front réutilise ce stockage pour rendre l'expérience plus rapide et résiliente si l'API news échoue temporairement.

7. Technologies utilisees et bilan du projet

7.1 Technologies - lecture approfondie

Technologie	Principe	Usage dans NewsHub	Exemple concret
Angular	Framework SPA structure en composants	Toutes les pages et composants reutilisables du projet	HomePageComponent, AuthCard, ProfilePageComponent
RxJS	Programmation reactive par observables	Gestion des flux utilisateur, requetes HTTP, refresh de session, debounce filtres	AuthService.currentUser\$, NewsService.getHomeFeed()
FastAPI	Framework Python rapide pour APIs	Expose les endpoints login, signup, favoris, premium, chatbot, live	backend/main.py
SQLAlchemy	ORM reliant objets Python et tables SQL	Modelise users, news, favorite, comments, live_events	backend/models.py + SessionLocal
JWT	Token signe qui transporte l'identite de session	Token de connexion stocke en local et verifie sur endpoints proteges	create_access_token(), verify_token()
MySQL	Base relationnelle durable	Persistence des utilisateurs, interactions et rooms live	database.py + init_db.sql
WebSocket	Canal bidirectionnel temps reel	Chat live, viewer count, signaling WebRTC	live_event_socket(), LiveRoomSocketService.connect()
WebRTC	Transport peer-to-peer audio/video	Diffusion camera editeur vers viewers	createOfferForViewer(), handleViewerOffer()
Ollama	Runtime local pour LLM	Assistant premium article-aware en local	simple_chatbot.py -> _ollama('/api/chat')
localStorage	Stockage navigateur persistant	Session utilisateur et cache de news	AuthService + NewsService.persistStore()

7.2 Explications approfondies par technologie

JWT

JWT sert a transporter une preuve d'authentification sans stocker l'etat serveur dans une session classique. Dans NewsHub, `create_access_token()` fabrique un token signe avec `SECRET_KEY` et une date d'expiration. Ensuite `AuthService.getToken()` le relit, verifie son expiration et invalide la session locale si le token n'est plus valable. L'interet pratique est fort : le backend peut reconnaitre l'utilisateur sur plusieurs routes sans garder une session memoire liee au navigateur.

SQLAlchemy

SQLAlchemy joue ici le role de pont entre les classes Python et MySQL. Par exemple `models.User`, `models.News` et `models.Favorite` representent directement les tables. Quand `save_favorite()` est execute, SQLAlchemy permet de rechercher une news, creer une relation favorite, puis `commit()` l'ensemble. Cette approche rend le code plus lisible qu'un grand nombre de requetes SQL ecrites a la main.

WebSocket

WebSocket est utilise quand une requete reponse classique ne suffit plus. Dans NewsHub, la room live a besoin de pousser des messages sans recharger la page : nombre de viewers, messages chat, updates editeur, signaling WebRTC. C'est exactement le travail de `live_event_socket()` cote backend et de `LiveRoomSocketService.connect()` cote frontend.

WebRTC

WebRTC ne remplace pas WebSocket, il le complete. WebSocket sert à dire qui parle à qui, tandis que WebRTC transporte la vidéo et l'audio entre pairs. NewsHub l'utilise dans LiveRoomPageComponent avec `createBroadcasterPeerConnection()`, `handleViewerOffer()` et les messages `offer/answer/ice_candidate`. C'est un choix efficace car la vidéo ne surcharge pas le backend.

FastAPI

FastAPI structure proprement les routes backend et les dépendances. L'application définit des endpoints courts, nommés et cohérents comme `/login`, `/favorites`, `/chatbot/status` ou `/live-events`. Sa force ici est la lisibilité et la rapidité de mise en place d'une API moderne avec typage via Pydantic.

RxJS

RxJS apporte un style réactif dans Angular. AuthService expose `currentUser$` pour que plusieurs pages se mettent à jour automatiquement quand la session change. NewsService emploie `shareReplay()` pour mettre en cache les news et éviter des appels répétitifs. Ce n'est pas seulement technique ; c'est une vraie optimisation de l'expérience utilisateur.

Ollama

Ollama fournit une exécution locale du modèle. Dans `simple_chatbot.py`, `_ollama()` appelle `/api/tags` pour vérifier les modèles installés puis `/api/chat` pour les réponses. Cela rend le projet autonome, sans dépendance à une API cloud payante. C'est très pertinent pour une application de démonstration locale.

MySQL

MySQL est la mémoire structurante du projet. Les comptes, favoris, commentaires et événements live doivent survivre aux rechargements de page, ce que `localStorage` seul ne peut pas faire. `database.py` configure la connexion, crée la base si nécessaire et ajoute même certaines colonnes manquantes pour faciliter l'évolution du schéma.

7.3 Technologies

- Angular : SPA, composants standalone, router, forms, reactive forms.
- RxJS : observables, subscriptions, debounce, finalize, shareReplay.
- FastAPI : exposition des routes REST et WebSocket.
- SQLAlchemy : mapping objet-relationnel et persistance MySQL.
- MySQL : stockage durable des comptes, interactions et rooms live.
- JWT : gestion des sessions et de l'authentification stateless.
- Ollama : exécution locale du modèle d'assistance IA.
- WebSocket + WebRTC : temps réel et vidéo peer-to-peer pour les lives.

7.4 Forces du projet

- Projet riche, couvrant beaucoup de besoins full-stack dans une seule application.
- Bonne séparation des responsabilités entre composants, services frontend et routes backend.
- Personnalisation utile grâce aux intérêts, au cache de news et aux parcours premium.
- Module live ambitieux avec vraie logique de signaling et streaming navigateur.
- Présence d'une sécurité minimum sérieuse pour un projet académique ou portfolio.

7.5 Points d'attention et améliorations possibles

- Uniformiser la configuration d'environnement pour eviter les URLs hardcodees comme http://127.0.0.1:8000 dans certains services.
- Rapprocher le brief documentaire du code reel pour lever l'ecart gemma3:12b / qwen3:14b.
- Renforcer les tests automatises frontend/backend, aujourd'hui peu visibles dans la structure.
- Ajouter une couche de gestion d'erreurs plus homogène sur tout le front.
- Prevoir une vraie pagination ou du lazy loading si le volume d'articles augmente.

Bilan final

NewsHub est un projet complet et pedagogiquement fort.

Il montre une vision produit claire, une architecture moderne et plusieurs niveaux d'interaction utilisateur.

Une personne qui lit ce rapport doit comprendre que le projet ne se limite pas a afficher des news: il orchestre de la personnalisation, de la securite, du premium, de l'IA et du live temps reel.

Fin du rapport.